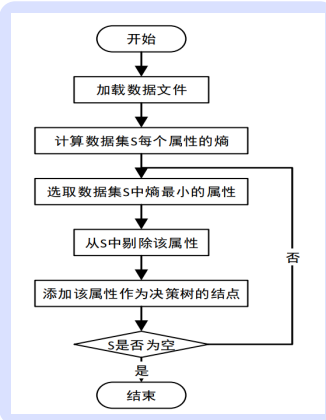


第四章 决策树算法

算法流程与实现



计算数据集S中每个属性的熵
选取数据集S中熵值最小（信息增益最大）的属性
在决策树上生成该属性结点
使用剩余重复结点重复以上步骤生成决策树的属性结点

算法关键问题

递归结束条件

叶子结点纯了——仅包含一类实例
达到最大深度
达到某一性能指标

连续属性离散化

分类数据有二元属性、标称属性等几种不同类型的离散属性
二元属性只有两个可能值，如“是”或“否”“对”或“错”，在分裂时，可以产生两个分支。对于二元属性，无需对其数据进行特别的处理
标称属性存在多个可能值，针对所使用的决策树算法的不同，标称属性的分裂存在两种方式：多路划分和二元划分，对于ID3、C4.5等算法，均采用多分支划分的方法，标称属性有多少种可能的取值，就设计多少个分支；CART算法采用二分递归分割的方法，因此该算法生成的决策树均为二叉树
标称属性中有类特别的属性为序数属性，其属性的取值是由先后顺序的。对于序数属性的分类，往往要结合实际情况来考虑

过拟合问题

模型的训练误差比较高，则称此分类模型欠拟合度

解决措施：增加分类属性的数量、选取合适的分类属性等方法，提高模型对于训练样本的拟合程度

模型的训练误差低但是泛化误差比较高，则称此分类模型过拟合

注意数据训练集的质量，选取具有代表性样本的训练样本集；另一方面避免决策树过度增长，通过限制树的深度来减小数据中的噪声影响（一般采用剪枝）

剪枝：缩小决策树的规模，从而降低算法复杂度并提高预测准确度

预剪枝：提前终止决策树的生长，在形成完全拟合训练样本集的决策树之前就停止树的生长，避免决策树规模过大而产生过拟合

后剪枝：先让决策树完全生长，之后针对子树进行判断，用叶子结点或者子树中最常用的分支替换子树，直到无法改进为止

错误率降低剪枝(REP)——后剪枝

从叶子结点向上，依次将决策树的所有子树用其样本中最多的类进行替换，记录下对于决策树的每颗子树剪枝前后的误差数之差，选取误差数减少最少的子树进行剪枝，选取误差数减少最少的子树进行剪枝，将其用子样本集中最大的类替换。按此步骤自底向上，遍历决策树的所有子树，当发现没有可替换的子树时，即每颗子树剪枝后的误差数都会增多，剪枝结束。

REP方法简单快速，在数据量较大时效果不错，但是由于需要比对模型子树替换前后的预测错误率，因此需要从数据集中划分出单独的测试集，故而当数据集较小时，REP剪枝策略的效果会下降

$$\alpha = \frac{R(t) - R(T_r)}{|N_t| - 1}$$

代价复杂度剪枝(CCP)

代价是指在剪枝过程中因为子树被替换而增加的错分样本，复杂度表示剪枝后减少的叶结点数
CCP使用α来衡量代价与复杂度之间关系的值，CCP的具体方法为，计算决策树的每个非叶子结点的α值，每次计算之后剪掉具有最小α值的子树，循环此过程直至只剩根结点，进行n次剪枝，生成n个决策树，从这n个决策树中根据真实误差估计选择最佳决策树

后剪枝决策树的欠拟合风险很小，泛化性能往往优于预剪枝决策树。但后剪枝训练时间比未剪枝和预剪枝决策树要大得多

算法改进与优化

使用增量训练的方式迭代训练决策树
融合Bagging与Boosting技术训练多颗决策树
对于波动不大、方差较小的数据集，可以探寻一种稳定的分裂准则作为解决方法

算法概述

树形结构，每个内部节点表示在一个属性上的测试，每个分支代表一个测试输出，每个叶节点代表一种类别

自顶向下的递归方法，其基本思想是以信息熵为度量构造一棵熵值下降最快的树，到叶子节点处的熵值为0.此时每个叶结点中的实例都属于同一类

决策树由决策结点、分支节点和叶子结点组成
决策节点表示在样本的某个属性上进行划分
分支表示对于决策节点进行划分的输出
叶结点代表经过分支到达的类

比较适合分析离散数据，如果是连续数据要先转换成离散数据再做分析

基本思路：
1.根据某种分类规则得到最优的划分特征，计算最优特征子函数，并创建特征的划分节点，根据划分节点建数据集划分为若干份子数据集
2.在子数据集上重复使用判别规则，构建出新的节点，作为树的新分支
3.重复递归执行，知道满足递归终止条件

算法基本思想

基于信息增益的ID3算法

以信息增益最大的属性为分类特征，基于弹性策略自顶向下搜索遍历决策空间，通过递归方式构建决策树

熵：表示随机变量不确定性度量（物体内部的混乱程度，越混乱，熵越高

熵的计算公式： $H(x) = -\sum_{i=1}^n p(x_i) \log_2 p(x_i)$
假设有一个数据集A，包含1-40的整数共40个
有一个叶子B，约占总概率50%，约含1-20的整数共20个
有一个叶子C，约占总概率50%，约含21-40的整数共20个

数字A： $-\left(\frac{1}{2} \times \log_2 \frac{1}{2}\right) \times 40 \approx 2.585$
数字B： $-\left(\frac{1}{2} \times \log_2 \frac{1}{2}\right) \times 20 - \frac{1}{2} \times \log_2 \frac{1}{2} \approx 2.161$
数字C： $-\left(\frac{1}{2} \times \log_2 \frac{1}{2}\right) \times 20 = 0$

信息熵的计算：

具体计算过程参考PPT

总结：
信息增益与属性的值域大小成正比。属性取值种类越多，越有可能成为分裂属性
ID3不能处理连续分布的数据特征
ID3算法对可取值值多的属性有偏好

优点：克服了信息增益偏向于选择特征值个数较多的不足
C4.5能够完成对连续属性的离散化处理，能够对不完整数据进行处理
分类规则易于理解、准确率较高
缺点：效率低，只适合于能够驻留于内存的数据集

基于信息增益率的C4.5算法

改进：C5.0 决策树构建速度加快、规模更小、拥有更少的叶子结点数
使用了提升法，组合多个决策树做出分类，使准确率大大提高
提供可选项由使用者视情况而定，例如是否考虑样本呢的权重、样本错误分类成本

基于基尼系数的CART 算法

天气与打球决策数据集					
Outlook	Temp	Humidity	Windy	Play	
1 Sunny	Hot	High	Weak	No	
2 Sunny	Hot	High	Strong	Yes	
3 Overcast	Hot	High	Weak	Yes	
4 Rain	Cool	High	Weak	Yes	
5 Rain	Cool	Normal	Weak	Yes	
6 Rain	Cool	Normal	Strong	No	
7 Overcast	Cool	Normal	Strong	Yes	
8 Sunny	Hot	Normal	Weak	Yes	
9 Sunny	Cool	Normal	Weak	Yes	
10 Rain	Hot	Normal	Weak	No	
11 Sunny	Hot	Normal	Strong	Yes	
12 Overcast	Hot	High	Strong	Yes	
13 Overcast	Hot	Normal	Weak	Yes	
14 Rain	Hot	High	Strong	No	

CART算法在分支属性中分支属性的信息增益使用基尼系数
 $Gini(D) = 1 - \sum_{i=1}^n p_i^2$
在数据集D中，假设属性A有m个取值，那么基尼系数为：
 $Gini(D) = 1 - \sum_{i=1}^m \left(\frac{|D_i|}{|D|}\right)^2$